TP3 : développer avec Codex et intégrer l'API dans une boutique

Durée : 50 min **Partie** : P3 (revue, débogage, tests, documentation et API)

Objectif

Piloter Codex dans VS Code : lui faire lire un projet, reproduire un bug, modifier les fichiers et lancer les tests. Relire ses changements, puis lui faire intégrer l'API OpenAI pour préparer la fiche d'un nouvel article de boutique.

Prérequis

- Démos 3 et 4 ; extension Codex installée et connectée au compte ChatGPT ; environnement commun préparé avec `uv sync`.
- Ouvrir `tps/tp3_codex_api/` seul dans VS Code.
- Ouvrir Codex par son icône ou par **Codex: Open Codex Sidebar** dans la palette de commandes. Travailler localement. Vérifier le contrôle des permissions sous la saisie : **Ask for approval** si disponible. Ce mode autorise les actions dans le dossier et demande avant de dépasser ce périmètre ; chaque commande locale ne déclenche pas nécessairement une demande.
- Lire `AGENTS.md`. Utiliser les données fictives fournies, conserver une copie de départ et ne pas modifier les tests d'acceptation ni consulter `corrige/` pendant le travail.
- Pour l'appel réel, clé de formation dans `OPENAI_API_KEY` (voir le README de la formation). Les tests et le client simulé permettent de poursuivre en cas de problème d'accès.

Partie 1 — Faire une revue ciblée (7 min)

Ouvrir `legacy/notes_frais.py` et `SPEC.md`. Avant de solliciter Codex, repérer le mot de passe fictif et remplacer sa valeur par `***`. Demander ensuite dans le panneau Codex :

> Lis AGENTS.md, SPEC.md et legacy/notes_frais.py. Sans modifier de fichier, relève les écarts à la spécification dans un tableau : ligne, règle concernée, gravité, correction proposée. Propose un plan court et indique les commandes pour reproduire puis vérifier les problèmes. Ne consulte pas corrige/.

Vérifier deux remarques dans les fichiers et ajuster le plan. Codex a accès au projet : vérifier les fichiers qu'il a lus et les actions qu'il annonce.

Partie 2 — Reproduire et corriger les écarts (12 min)

> Exécute uv run python legacy/notes_frais.py donnees/notes_septembre.csv et uv run pytest -q test_acceptation_tp3.py. Explique les échecs observés par rapport à SPEC.md, puis applique les corrections du plan dans legacy/notes_frais.py uniquement. Retire le mot de passe inutilisé, remplace eval, puis relance ces commandes. Ne modifie pas les tests fournis.

Lire les commandes et leurs sorties dans la conversation. Si une autorisation est demandée, vérifier sa cible avant de l'accorder. Pour remplacer `eval`, ce modèle peut aider Codex :

```
import json
from pathlib import Path

fichier_taux = Path(__file__).resolve().parent.parent / "taux.json"
with fichier_taux.open(encoding="utf-8") as fichier:
    taux = json.load(fichier)
```

Faire corriger la conversion avant plafond, la virgule décimale, le rejet signalé d'une catégorie inconnue et l'affichage français. Pour traiter un montant, utiliser ce point de départ :

```
montant = float(texte.strip().replace(",", "."))
en_euros = montant * taux[devise]
# Appliquer ensuite le plafond de la catégorie, puis arrondir au centime.
```

Ouvrir les différences proposées par Codex et expliquer une correction avant de la conserver. Relancer soi-même `uv run pytest -q test_acceptation_tp3.py` dans le terminal. Les totaux attendus sont Diallo 48,92 EUR, Martin 180,30 EUR et Nguyen 251,40 EUR ; la dépense « taxi » est rejetée avec un motif.

Partie 3 — Écrire les tests depuis la spécification (8 min)

> À partir des règles et exemples de SPEC.md, propose d'abord un tableau de cas et de résultats attendus. Après ma relecture, crée test_notes_frais.py avec ces tests pytest, adapte les appels aux fonctions corrigées et exécute les tests. Ne déduis pas les valeurs attendues du code et ne change pas les tests fournis.

Vérifier à la main trois cas avant la création du fichier : conversion avant plafond, montant à virgule et catégorie inconnue.

```
# Exemple d'attendu tiré des règles, à adapter au nom de votre fonction :
assert montant_rembourse("hotel", 110, "GBP", {"GBP": 1.17}) == 120.0
```

Lancer `uv run pytest -q test_acceptation_tp3.py test_notes_frais.py`.

Partie 4 — Documenter le résultat (3 min)

Demander à Codex d'ajouter les docstrings et de créer `README.md` avec une section « Utilisation » de dix lignes maximum. Lui demander de vérifier la commande et la sortie en exécutant le script, sans inventer d'option. Relire les fichiers modifiés.

Partie 5 — Générer la fiche d'un nouvel article (15 min)

La boutique possède `catalogue.json`. À l'ajout de `nouvel_article.json`, elle prépare sa fiche en brouillon. Les fiches existantes doivent rester intactes ; une référence déjà présente est refusée avant l'appel API. La démo 4 a montré le même mécanisme d'intégration sur le tri d'un ticket support.

1. Lire `ArticlePublic`, `FicheProduit` et les deux fonctions à compléter dans `catalogue.py`.
2. Demander à Codex de compléter uniquement `generer_fiche` et `ajouter_article`, puis de lancer `uv run pytest -q test_api_catalogue.py` avec le client simulé, sans appel réel. Le client est reçu en paramètre. Le bloc suivant montre la structure de l'appel ; expliquer chaque argument avant de le garder :

```
reponse = client.responses.parse(
    model=modele,
    instructions=INSTRUCTIONS,
    input=json.dumps(donnees, ensure_ascii=False),
    text_format=FicheProduit,
    store=False,
)
if reponse.output_parsed is None:
    raise ValueError("Aucune fiche structurée : refus ou réponse incomplète.")
return FicheProduit.model_validate(reponse.output_parsed)
```

3. Relire les changements de Codex, notamment la fin d'`ajouter_article`, après le contrôle des doublons. Exemple de construction :

```
fiche = generer_fiche(public, client, modele)
nouveau = {**public, "fiche": fiche.model_dump(), "statut": "brouillon"}
return [*catalogue, nouveau]
```

4. Relancer soi-même `uv run pytest -q test_api_catalogue.py` : les huit tests utilisent un client simulé. Avec la clé, lancer dans son terminal `uv run python catalogue.py catalogue.json nouvel_article.json`, sans transmettre la clé dans la conversation Codex. Lire `catalogue_brouillon.json` et comparer chaque affirmation aux caractéristiques. En cas d'accès API indisponible, vérifier le même parcours avec les tests et le formateur. Aucun site n'est publié par cet exercice.

Partie 6 — Vérifier et expliquer (5 min)

Lancer soi-même `uv run pytest -q`. Montrer une correction de Codex avec sa règle et son test, puis un changement que l'on a vérifié ou fait reprendre. Expliquer ce qui déclenche l'appel API, ce qui est envoyé et pourquoi le texte généré reste à relire.

Critères de réussite

- Les 15 tests fournis passent : 7 pour les notes de frais et 8 pour le catalogue ; vos tests passent aussi.
- Le README décrit une commande et une sortie vérifiées.
- Le nouvel article reçoit une fiche en brouillon ; les anciens articles sont conservés ; un doublon ne provoque aucun appel.
- Vous avez vérifié les commandes exécutées et les différences produites par Codex.
- Vous distinguez Codex, qui développe le projet, et l'API, que l'application appelle pour générer une fiche.

Corrigé

`corrige/legacy/notes_frais.py`, `corrige/catalogue.py`, `corrige/test_notes_frais.py`, `corrige/README.md` et `corrige/prompts_types.md`.